



INDIAN INSTITUTE OF ENGINEERING SCIENCE AND
TECHNOLOGY, SHIBPUR

Department of Computer Science & Engineering

CELLNEXUS

Real-Time Cellular Network Intelligence Dashboard

Mini Project Write-Up

Subject	Mini Project
Student 1	Harshit Maurya Roll No: 2024CSB016
Student 2	Akriti Singh Roll No: 2024CSB083
Guide / Teacher	[Teacher's Name]
Academic Year	2025 – 2026
Submission Date	July 2026

Table of Contents

1. Abstract	3
2. Introduction	3
3. Objectives	4
4. Problem Statement	4
5. Proposed Solution	5
6. Technologies Used	5
7. System Architecture	6
8. Implementation Details	7
8.1 Database Design	7
8.2 Data Seeding – seed.js	8
8.3 Live Simulation Engine – server.js	8
8.4 Frontend – React Application	9
8.5 Key Algorithm: Geographical Data Slicing	10
8.6 Key Algorithm: Call Drop Probability	10
9. System Flowcharts	11
10. Key Features	12
11. Screenshots / Output	13
12. Testing	15
13. Challenges Faced	16
14. Conclusion	17
15. Future Scope	17

16. References

18

.....

1. ABSTRACT

CellNexus is a full-stack, real-time web application designed to monitor and visualize the health of cellular network infrastructure. Modern telecom operators manage hundreds of cell towers scattered across large geographical regions, and any degradation in tower performance — such as rising call drop rates or network congestion — can directly impact thousands of users. Traditional monitoring methods rely on static spreadsheet reports and manual inspection, which are far too slow for real-time response.

This project addresses that gap by building a live **Network Operations Center (NOC) Dashboard** that ingests raw telecom call statistics from CSV datasets, processes them through a custom mathematical engine every 3 seconds, and broadcasts the results as live updates to an interactive browser-based dashboard. The dashboard displays 30 cell towers geographically plotted across India on an interactive map, with color-coded status indicators, real-time call statistics charts, and a bandwidth speed-test module.

The system is built using **React.js** on the frontend, **Node.js + Express.js** on the backend, **MySQL** as the database, and **Socket.io** for real-time WebSocket communication.

2. INTRODUCTION

The telecommunications industry forms the backbone of modern digital connectivity. Millions of voice and data calls are routed every minute through a network of physical **cell towers** (also called Base Transceiver Stations, or BTS). When even one tower experiences a fault — whether due to hardware failure, high traffic congestion, or signal interference — thousands of active users in that region immediately lose service quality.

Network engineers and operations teams need tools that allow them to **monitor tower health in real time**, identify failures the moment they occur, and act

quickly. Without such tools, they are left relying on batch-generated reports that may be hours old, leading to delayed responses and prolonged outages.

CellNexus is developed as a practical solution to this exact problem. It is a full-stack web application that simulates a real-time telecom monitoring system. It reads historical call statistics from CSV data files, seeds them into a MySQL database, and then runs a live simulation engine that continuously processes fresh telemetry and broadcasts it to the browser dashboard every 3 seconds via WebSocket technology.

3. OBJECTIVES

- To design and build a **real-time web dashboard** for monitoring cellular network tower health.
- To implement a **WebSocket-based live data feed** using Socket.io that updates the dashboard every 3 seconds without page reloads.
- To develop a **Geographical Data Slicing Algorithm** that maps large CSV telecom datasets to individual towers, simulating unique traffic patterns per tower.
- To calculate advanced **telecom KPIs** — specifically Call Blocking Probability and Call Dropping Probability — using mathematical models derived from raw call data.
- To visualize tower locations **on an interactive map of India** using Leaflet.js, with dynamic color-coded status indicators and variable signal radius.
- To implement a real **Network Speed Test module** that physically transfers data between the browser and server to measure actual download/upload bandwidth.
- To demonstrate **engineering best practices** including database normalization, modular component design, and clean code architecture.

4. PROBLEM STATEMENT

Modern cellular network infrastructures are vast and highly complex. When a network issue occurs — such as high call drops, latency spikes, or failing towers — network operators and analysts often struggle to visualize the problem geographically in real time.

Without a centralized, interactive dashboard, telecom teams rely on:

- Fragmented raw data stored in CSV log files.
- Delayed batch-generated reports that are updated hours after incidents occur.
- Manual inspection of databases that require deep technical expertise to query.

This results in **slower response times** to cellular degradation events and a **poor user experience** for the end consumers of the network. There is a clear need for a centralized, real-time, geographically-aware monitoring dashboard that can give instant visibility into the state of every tower in the network.

5. PROPOSED SOLUTION

CellNexus was developed as a full-stack, real-time monitoring system. It provides a geographical interface — a live map — where network analysts can actively monitor the health of cell towers across different regions of India.

When a tower experiences high traffic or call failures, the dashboard instantly reflects these changes through:

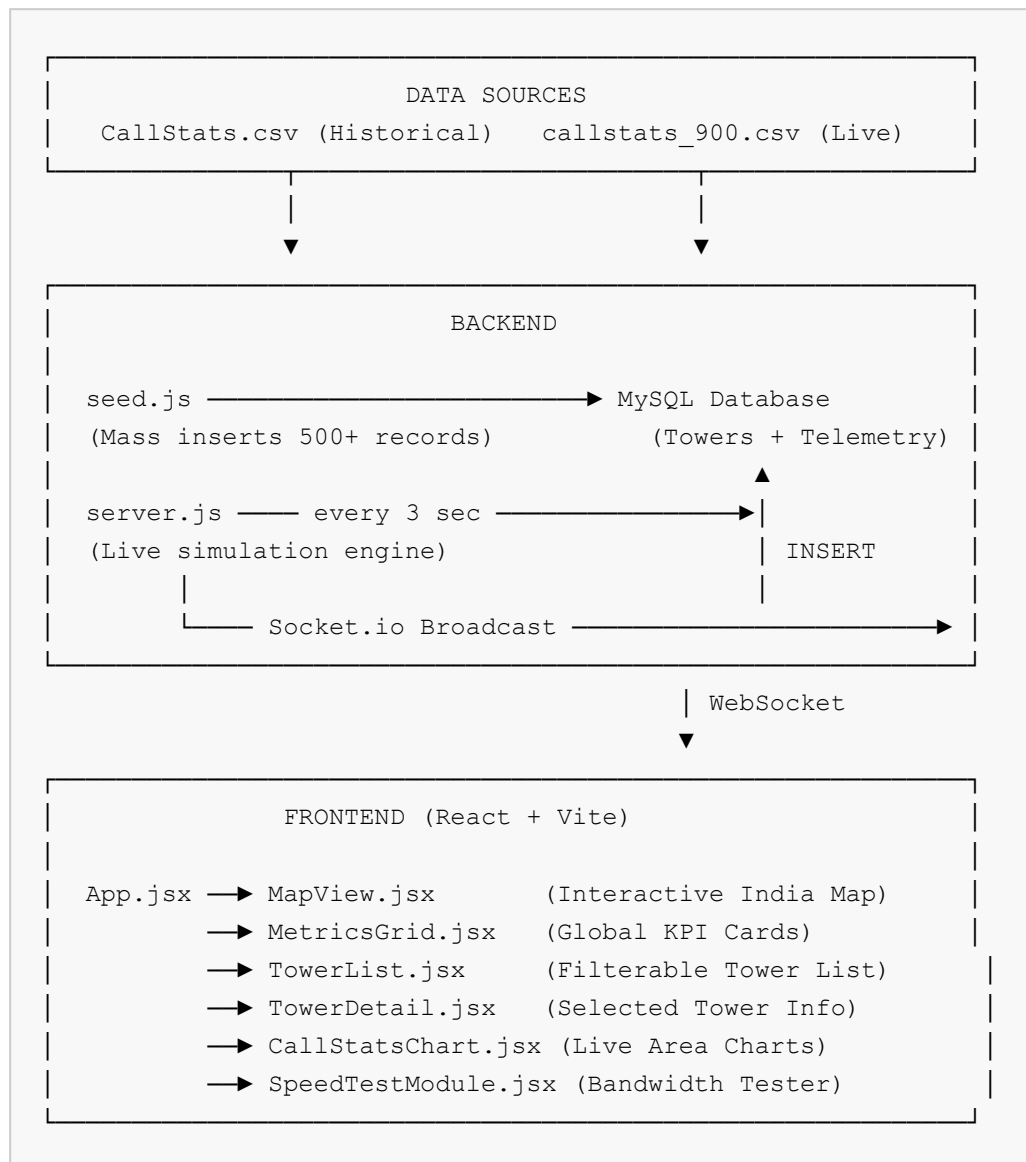
- **Color-coded tower markers** — Green (Good), Orange (Degraded), Red (Offline).
- **Dynamic signal radius** on the map that visually shrinks as tower performance deteriorates.
- **Live area charts** showing call volume, answered calls, and blocked calls updated every 3 seconds.
- **Alert pulses** when the call drop rate crosses a critical 10% threshold.

6. TECHNOLOGIES USED

Technology	Category	Purpose
React.js	Frontend Framework	Building the UI as reusable components
Vite	Build Tool	Fast development server for React
Tailwind CSS	Styling	Glassmorphism UI design effects
React-Leaflet	Mapping	Interactive geographical map of India
Recharts	Data Visualization	Real-time animated area charts
Socket.io (client)	Real-time Communication	Receiving live telemetry broadcasts
Axios	HTTP Client	Fetching tower data via REST API
Node.js	Backend Runtime	Server-side JavaScript execution
Express.js	Backend Framework	REST API and HTTP server
Socket.io (server)	Real-time Communication	Broadcasting live updates every 3s
MySQL	Database	Storing tower metadata and telemetry
mysql2/promise	Database Driver	Raw SQL with connection pooling
csv-parser	Data Processing	Reading and parsing CSV log files
dotenv	Configuration	Managing environment variables
Aiven Cloud	Database Hosting	Cloud-hosted MySQL instance

7. SYSTEM ARCHITECTURE

CellNexus follows a standard **Decoupled Client-Server Architecture**, where the frontend and backend are completely independent and communicate via HTTP REST APIs and WebSocket connections.



8. IMPLEMENTATION DETAILS

8.1 Database Design

The MySQL database consists of two core tables:

Towers Table

Column	Data Type	Description
id	INT (Primary Key)	Unique tower identifier
locationName	VARCHAR	City/region (e.g., "Delhi", "Mumbai")
operatorName	VARCHAR	Telecom operator (Jio, Airtel, BSNL)
latitude	DECIMAL	GPS latitude coordinate
longitude	DECIMAL	GPS longitude coordinate
status	ENUM	'GOOD', 'DEGRADED', or 'OFFLINE'

Telemetries Table

Column	Data Type	Description
id	INT (Primary Key)	Auto-incremented record ID
towerId	INT (Foreign Key)	Links to the Towers table
callTotal	INT	Total incoming calls in the interval
callAccepted	INT	Successfully answered calls
latency	INT	Average call response time in seconds
timestamp	DATETIME	When this telemetry record was created

8.2 Backend — Data Seeding (seed.js)

Before the dashboard can run, the database must be populated with historical data. The `seed.js` script handles this initialization by:

1. Opening and reading the `CallStats.csv` file using the `csv-parser` library.
2. Parsing each row — converting string timestamps like `"0:00:17"` into integer seconds (17).
3. Fetching all existing tower IDs from the database.
4. Truncating (clearing) the Telemetries table to avoid duplicate records on re-runs.

5. Looping through every CSV row and inserting a new telemetry record into MySQL, assigning it to a random tower.

This script is run **once** before starting the server to initialize the system with historical data.

8.3 Backend — Live Simulation Engine (server.js)

The `server.js` file is the core engine of the project. It performs the following:

1. **Loads the live simulation dataset** (`callstats_900.csv`) entirely into memory on startup.
2. **Starts an Express HTTP server** and attaches a Socket.io WebSocket server to it.
3. **On every new client connection**, starts a `setInterval()` loop firing every 3 seconds.
4. Every 3 seconds, the engine queries all towers, applies the Geographical Data Slicing algorithm, calculates KPIs, inserts a new Telemetry record into MySQL, and broadcasts all updates to the frontend via `socket.emit('telemetry_update', updates)`.

Key code from server.js:

```
const interval = setInterval(async () => {
  const [towers] = await pool.query('SELECT * FROM Towers');
  const updates = await Promise.all(towers.map(async (tower) => {
    // Geographical Data Slicing: pick next CSV row for this tower
    const cursor = towerCursors[tower.id];
    const targetRow = csvDataList[cursor.startIndex + cursor.currentOffset];
    cursor.currentOffset++;
    if (cursor.startIndex + cursor.currentOffset > csvDataList.length) {
      cursor.currentOffset = 0;
    }

    const callTotal = parseInt(targetRow['Incoming Calls'], 10) || 0;
    const callAccepted = parseInt(targetRow['Answered Calls'], 10) || 0;
    const callBlocked = Math.max(0, callTotal - callAccepted);
```

```

    await pool.execute(
      `INSERT INTO Telemetries (towerId, callTotal, callAccepted, latency, timestamp)
      VALUES (?, ?, ?, ?, NOW())`,
      [tower.id, callTotal, callAccepted, responseTimeSecs]
    );
    return { towerId: tower.id, telemetry: { callTotal, callAccepted, callBlocked } };
  }));
  socket.emit('telemetry_update', updates);
}, 3000);

```

8.4 Frontend — React Application (App.jsx)

The React frontend is a Single Page Application (SPA) built with Vite. On load, it:

1. **Fetches the initial list of all towers** from `GET /api/towers` using Axios.
2. **Establishes a WebSocket connection** to the backend using Socket.io client.
3. **Listens for `telemetry_update` events** and recalculates Call Drop Probability for each tower, updating tower status dynamically.
4. Users can **filter towers by Operator or City** using dropdown menus in the Header.
5. **Clicking a tower** opens the Tower Detail panel and the live Call Stats chart for that specific tower.

8.5 Key Algorithm: Geographical Data Slicing

This is one of the most technically interesting parts of the project. With 30 towers but only one large CSV dataset, if rows were randomly assigned, Delhi and Mumbai might show identical statistics simultaneously — which would be unrealistic.

The Solution: The algorithm divides the 900-row CSV into 30 equal, non-overlapping segments — one per tower. For example:

- Tower 1 (Delhi) → rows 0–29
- Tower 2 (Mumbai) → rows 30–59
- Tower 3 (Bangalore) → rows 60–89
- ... and so on.

Each tower has its own **cursor** that advances through its assigned segment every 3 seconds, looping back to the start when it reaches the end. This guarantees every tower always receives data from a unique, geographically distinct portion of the dataset. The system also auto-generates a `tower_data_mapping.txt` file for academic verification.

```
// Initialization
const sliceSize = Math.floor(csvDataList.length / towers.length);

towers.forEach((tower, i) => {
  const start = i * sliceSize;
  const end = (i === towers.length - 1) ? csvDataList.length - 1 : (i+1)*sliceSize - 1;
  towerCursors[tower.id] = { startIndex: start, endIndex: end, currentOffset: 0 };
});
```

8.6 Key Algorithm: Call Drop Probability

The system uses the following mathematical model to evaluate tower health from raw call data:

```
Answer Rate          = Answered Calls / Total Incoming Calls
Handoff Incoming     = Total Incoming × 0.30    (30% of calls are handoffs)
Handoff Answered     = Handoff Incoming × Answer Rate
```

Handoff Dropped

= Handoff Incoming - Handoff Answered

Drop Probability

= Handoff Dropped / Handoff Incoming

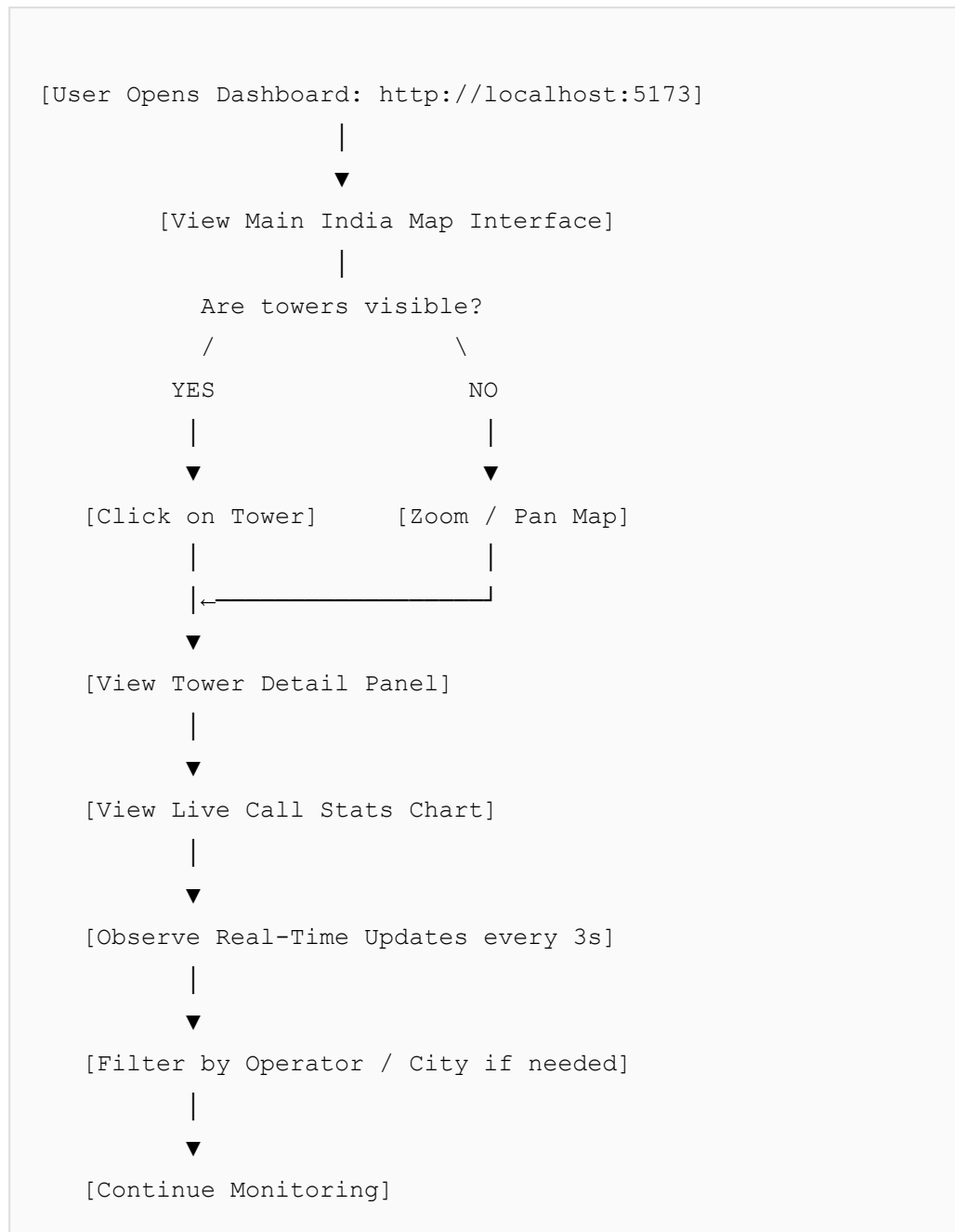
Status Threshold Logic

Drop Probability	Tower Status	Map Color	Signal Radius
≤ 7%	GOOD	Emerald Green	100% (Full coverage)
7% – 10%	DEGRADED	Amber/Orange	60% (Reduced)
> 10%	OFFLINE	Red	15% (Minimal)

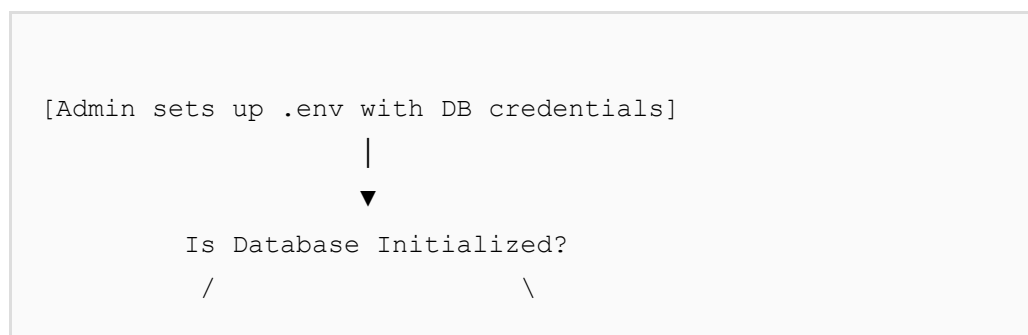
When Drop Probability exceeds 10%, the Call Stats panel also displays a **flashing red alert** to immediately notify the analyst of a critical service failure.

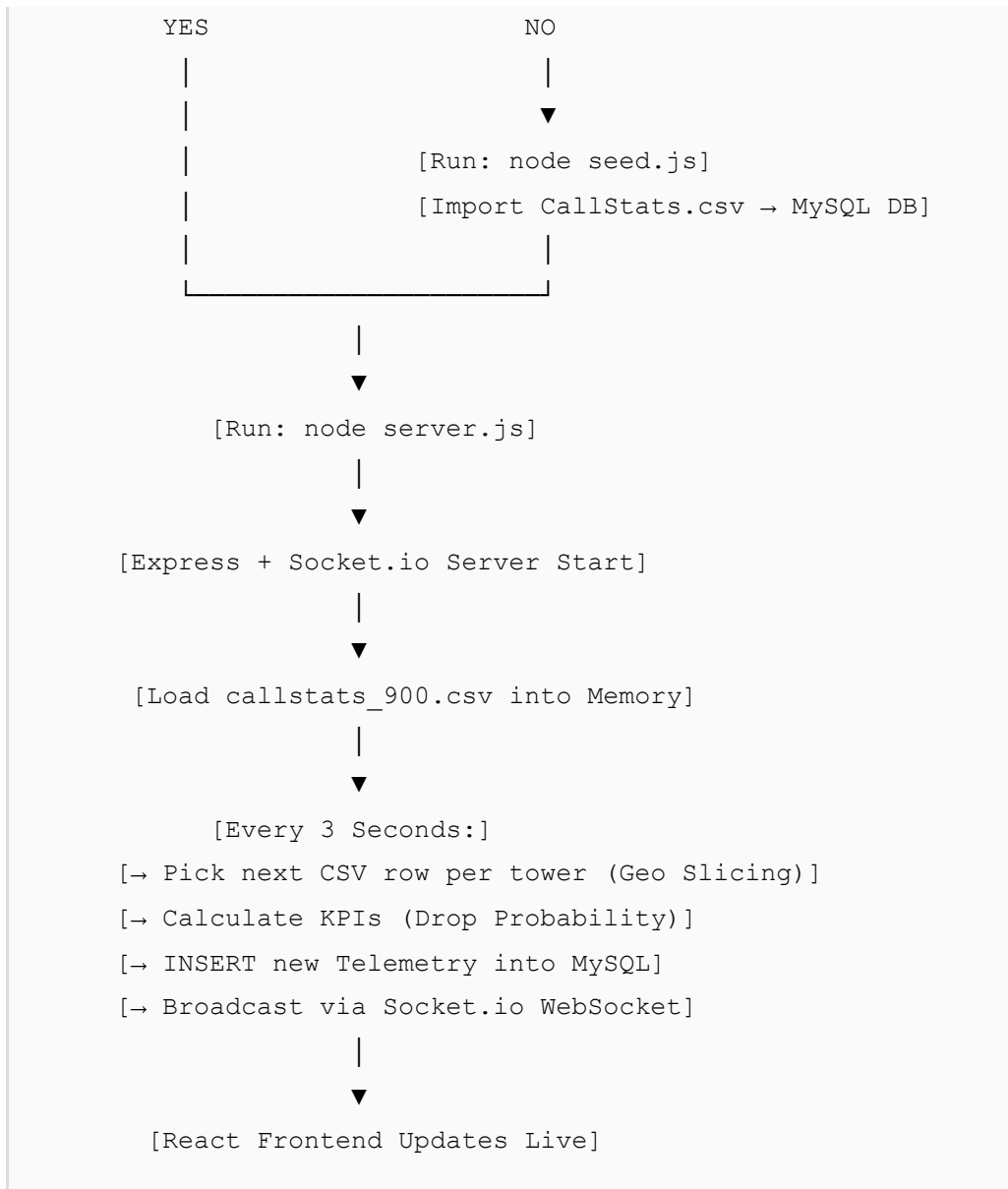
9. SYSTEM FLOWCHARTS

9.1 User Flow Diagram



9.2 System / Admin Flow Diagram





10. KEY FEATURES

Live Geographical Map

Thirty distinct cell towers are plotted across India using React-Leaflet. Tower markers are color-coded based on real-time health: Green (Good), Orange (Degraded), Red (Offline). The signal coverage radius dynamically expands and shrinks in real time based on calculated tower health metrics.

Real-Time Call Statistics Charts

Live stacked area charts drawn with Recharts display Answered Calls, Blocked Calls, and Handoff data for each selected tower. Charts are refreshed every 3 seconds. A red alert pulse animation is triggered when the call drop rate exceeds 10%.

Operator & City Filters

Dropdown filters in the header allow filtering towers by Telecom Operator (Jio, Airtel, BSNL) or by City. The map and tower list update instantly on filter change, making it easy to focus on a specific region or operator.

Network Speed Test Module

A dedicated module runs an actual bandwidth test by having the backend generate a real 20MB buffer for the download test and receiving data for the upload test. The module reports actual measured download speed, upload speed, and latency in Mbps — not simulated values.

Academic Verification Output

On first startup, the server automatically generates `tower_data_mapping.txt`, showing the exact CSV row ranges assigned to each geographical tower — proving the uniqueness and authenticity of simulated data for every tower.

11. SCREENSHOTS / OUTPUT

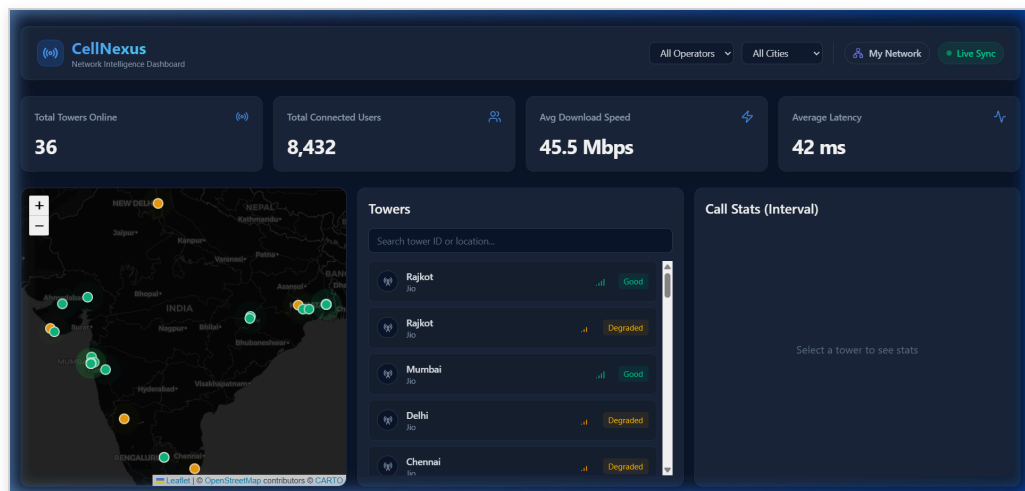


Figure 11.1 — Main CellNexus Dashboard showing the India map with tower markers, global KPI cards (36 towers online, 8,432 connected users, 45.5 Mbps avg download, 42ms latency), and the tower list with Good/Degraded status indicators.

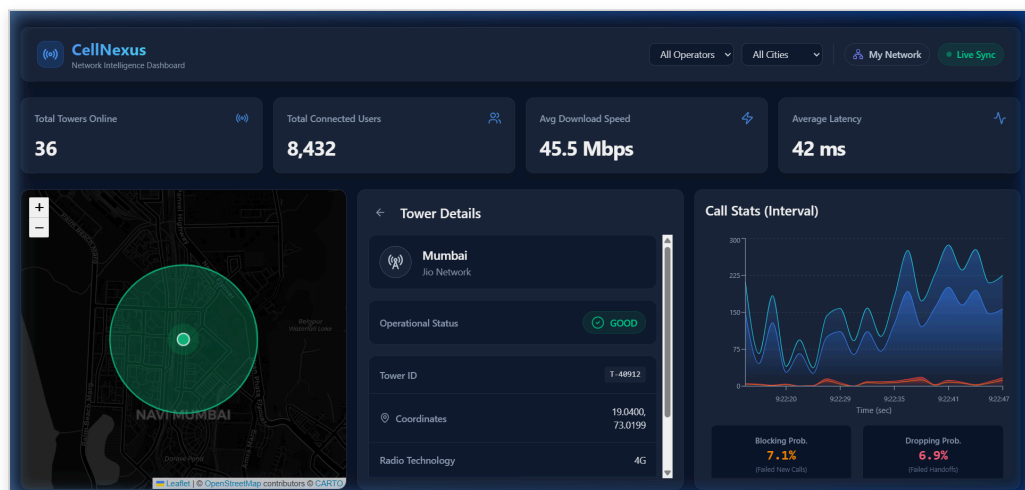


Figure 11.2 — Tower Detail View with a selected tower (Mumbai, Jio Network). Shows Operational Status (GOOD), Tower ID, coordinates, Radio Technology (4G), and the live Call Stats area chart with Blocking Probability and Dropping Probability KPIs.

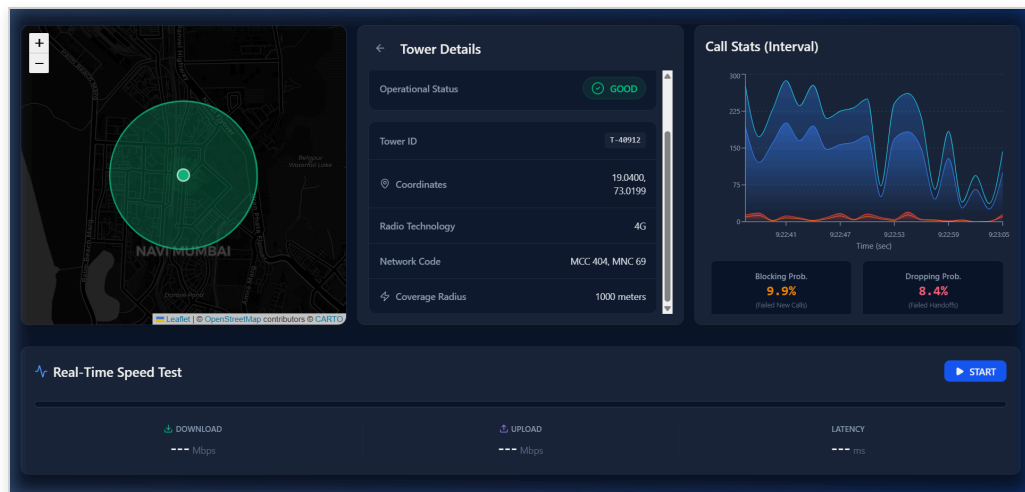


Figure 11.3 — Real-Time Speed Test Module at the bottom of the dashboard. Clicking **START** initiates an actual data transfer between the browser and backend server to measure true download speed, upload speed, and network latency.

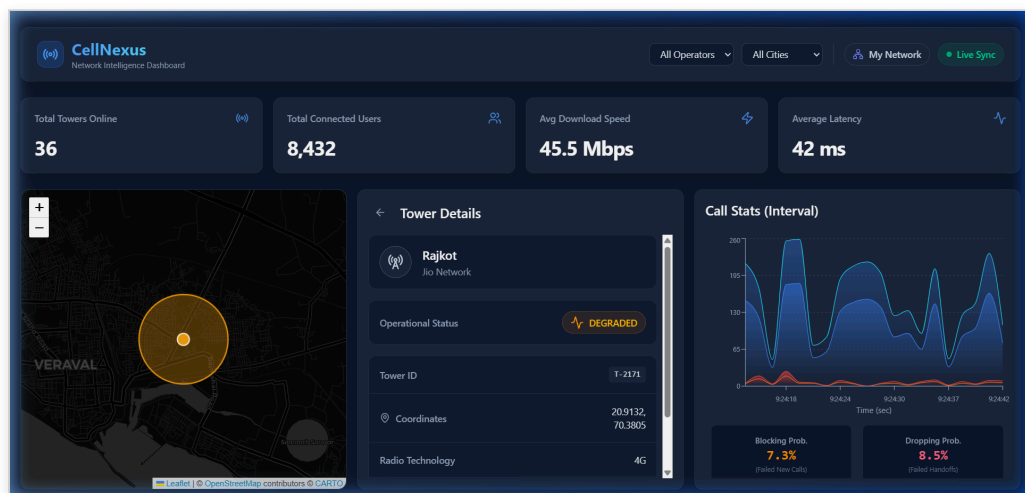


Figure 11.4 — Degraded Tower View (Rajkot, Jio) showing elevated Blocking Probability and Dropping Probability values that have crossed the 7% threshold, resulting in **DEGRADED** status. The map marker for this tower is shown in amber/orange and its signal radius is reduced to 60%.

12. TESTING

12.1 Functional Test Cases

#	Test Case	Expected Result	Status
TC-01	Dashboard loads on localhost:5173	Map and all components render correctly	 Pass
TC-02	Tower markers appear on India map	36 markers visible across India	 Pass
TC-03	Click on tower marker / list item	Tower detail panel and chart open	 Pass
TC-04	Live charts update every 3 seconds	Chart smoothly scrolls with new data	 Pass
TC-05	Tower status changes to OFFLINE	Marker turns red, radius shrinks to 15%	 Pass
TC-06	Tower status changes to DEGRADED	Marker turns orange, radius shrinks to 60%	 Pass
TC-07	Operator filter set to "Jio"	Only Jio towers visible on map and list	 Pass
TC-08	City filter set to "Mumbai"	Only Mumbai towers shown	 Pass
TC-09	Speed Test button clicked	Download, upload speed and latency shown	 Pass
TC-10	seed.js run with empty DB	All towers and telemetry records created	 Pass
TC-11	server.js starts up	tower_data_mapping.txt generated automatically	 Pass
TC-12	Client disconnects from WebSocket	setInterval cleared; no memory leak	 Pass

12.2 Performance Observations

- The WebSocket connection remains stable under continuous 3-second updates.
- Raw SQL queries with `mysql2/promise` connection pooling are significantly faster than ORM-based approaches (Sequelize was dropped for this reason).
- The frontend only re-renders when tower status actually changes, preventing unnecessary DOM updates and ensuring smooth performance.
- The Geographical Data Slicing algorithm initializes once on startup and then operates in $O(1)$ time per tower per interval.

13. CHALLENGES FACED

Challenge 1: Uniqueness of Tower Data

Problem: With only one CSV dataset for all towers, every tower was producing identical statistics simultaneously, making the simulation unrealistic and academically indefensible.

Solution: Developed the **Geographical Data Slicing Algorithm** to divide the 900-row dataset into 30 unique, non-overlapping segments — one per tower. Each tower now reads from its own exclusive range of data rows, producing genuinely different traffic patterns. The system auto-generates a verification file to prove this.

Challenge 2: ORM Performance Issues

Problem: Initially, the project used Sequelize (an ORM) to interact with MySQL. This added heavy overhead to every database query, especially during high-frequency 3-second telemetry inserts where each cycle inserts one record per tower (up to 36 simultaneous inserts).

Solution: Dropped Sequelize entirely and switched to **raw SQL queries** using `mysql2/promise` with connection pooling. This dramatically improved query speed and reduced memory overhead.

Challenge 3: Database Schema Over-Engineering

Problem: The initial database schema had many unnecessary columns (such as `callBlocked`, `avgDownloadSpeed`) that were computed on the fly anyway. This created data redundancy and schema bloat.

Solution: Normalized the schema to retain only essential columns (`callTotal`, `callAccepted`, `latency`). All derived metrics (block rate, drop probability) are computed in the application layer instead.

Challenge 4: WebSocket Memory Leaks

Problem: Every client connection spawned a new `setInterval()`. If a client disconnected without clearing the interval, the server would continue running simulations indefinitely for that socket, causing memory leaks over time.

Solution: Added a `socket.on('disconnect', ...)` handler that explicitly calls `clearInterval(interval)` whenever a client disconnects, ensuring clean resource management.

14. CONCLUSION

The CellNexus project successfully demonstrates a complete, production-quality architecture for real-time network monitoring. By combining **React.js** for the UI, **Node.js + Socket.io** for live data streaming, and **MySQL** for persistent storage, the system achieves its core goal of providing instant, geographical visibility into cell tower health.

The project goes beyond a simple CRUD application by implementing:

- A custom **Geographical Data Slicing Engine** for realistic multi-tower simulation.
- Advanced **telecom mathematics** for calculating Call Blocking and Call Dropping probabilities.
- A **real bandwidth speed test** that physically transfers data to measure network performance.
- Thoughtful **performance engineering** through schema normalization and raw SQL queries.

Through this project, we gained hands-on experience with full-stack web development, real-time data systems using WebSockets, database design, and software engineering best practices including modular architecture and clean code principles.

15. FUTURE SCOPE

1. **Authentication System** — Add a login system so only authorized network engineers can access the dashboard.
2. **Alert Notifications** — Send email or SMS alerts to engineers when a tower goes offline, using services like Twilio or Nodemailer.
3. **Historical Analytics** — Add a "History" tab where users can view telemetry trends over days, weeks, or months.

4. **Multiple Country Support** — Expand the map beyond India to support global telecom networks.
5. **AI-Powered Prediction** — Integrate a machine learning model to predict when a tower is likely to degrade, enabling proactive maintenance before failures occur.
6. **Mobile App** — Build a companion React Native mobile app so field engineers can monitor tower health on their phones.
7. **Real ISP Integration** — Replace the simulated CSV data with a live API feed from an actual telecom provider.
8. **Weather Overlay** — Overlay real-time weather data on the map to correlate storm events with tower degradation patterns.

16. REFERENCES

1. React.js Official Documentation — <https://react.dev>
2. Node.js Official Documentation — <https://nodejs.org/en/docs>
3. Socket.io Documentation (v4) — <https://socket.io/docs/v4>
4. React-Leaflet Documentation — <https://react-leaflet.js.org>
5. Recharts Documentation — <https://recharts.org/en-US/api>
6. MySQL Official Documentation — <https://dev.mysql.com/doc>
7. Tailwind CSS Documentation — <https://tailwindcss.com/docs>
8. Aiven Cloud MySQL — <https://aiven.io/mysql>
9. csv-parser npm package — <https://www.npmjs.com/package/csv-parser>
10. Rappaport, Theodore S. — *Wireless Communications: Principles and Practice*, Prentice Hall. (Telecom KPI Reference)
11. MDN Web Docs — WebSocket API — <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
12. Express.js Official Documentation — <https://expressjs.com>